

Deep Learning for NLP

Student name: *Angelos Dorotheos Chatzopoulos*
cs: 7115112400027

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Fall Semester 2025*

Contents

1	Abstract	3
2	Data Analysis and Preprocessing	3
2.1	Data initial analysis	3
2.1.1	Dimensions of datasets and checking for NULL or duplicated rows	3
2.1.2	Distribution of sentiment labels	3
2.1.3	Distribution of sentiment labels	4
2.1.4	Tweets length analysis before preprocessing	5
2.1.5	Finding top 10 unigrams, bigrams & trigrams before preprocessing	6
2.1.6	Counting emails and user mentions in tweets	6
2.2	Tweets preprocessing & further analysis	7
2.2.1	Tweets length after preprocessing with stop words filtering enabled	9
2.2.2	Distribution of sentiment labels after applying preprocessing . .	9
2.2.3	Finding top 10 unigrams, bigrams & trigrams after applying pre- processing	10
3	Preprocessing and Tokenization Configurations	13
3.1	Dataset preprocessing configurations	13
3.2	Tokenization experiments	13
3.3	Performance evaluation metrics	14
4	Experiments on BERT and DistilBERT Language Models	14
4.1	Preprocessing comparison: Stopwords filtering and slang words conver- sion	15
4.2	Tokenization max length tuning	19
4.3	Optimizer tuning	22
4.4	Optimizer scheduler testing	22
4.5	Optuna study	26
5	Final Results	27
5.1	BERT final model results	27
5.2	DistilBERT final model results	27
5.3	Comparison with the first project	29
5.4	Comparison with the second project	30

5.5	Future work	30
6	Bibliography	30

1. Abstract

This project develops a sentiment analysis classifier using pretrained language models (BERT and DistilBERT) on an English-language Twitter dataset. The dataset consists of three features: ID, text, and label, where the label indicates the sentiment of each tweet as positive (1) or negative (0). As in the first and second projects, exploratory data analysis (EDA) is performed on the train and validation sets to understand the distribution of labels and the characteristics of the tweets, especially due to changes in the preprocessing stage. Based on the analysis from EDA, text preprocessing is applied to remove unnecessary parts from the tweets. The cleaned texts are then tokenized using the corresponding tokenizers for BERT and DistilBERT. Using PyTorch, each model is loaded, and experiments are conducted to tune the preprocessing, tokenization and hyperparameters. Finally, by using multiple performance metrics such as accuracy, precision, recall, F1-score, as well as test and validation set loss, the final hyperparameters are selected for the final pretrained models.

Model	Kaggle Notebook URL
BERT	BERT Notebook
DistilBERT	DistilBERT Notebook

Table 1: Kaggle notebook links for BERT and DistilBERT models

2. Data Analysis and Preprocessing

2.1. Data initial analysis

Initially, similar to the first two projects, an in-depth analysis is performed to find the key characteristics of the training and validation datasets [10]. **Since a slightly different preprocessing approach was used from the previous two projects, the exploratory data analysis steps are redone (although it was not necessary for this homework).** This exploratory data analysis (EDA) phase plays a significant role in understanding the structure of our datasets. It allowed us to understand the distribution of sentiment labels, the tweet lengths, the existence of usernames and email addresses within the texts, as well as identifying the most frequently occurring tokens. Visual tools such as histogram plots and word clouds were utilized to guide us through the preprocessing and modeling stages afterwards.

2.1.1. Dimensions of datasets and checking for NULL or duplicated rows. In this step, we examine the dimensions of the given datasets. Table 2 shows the dimensions of the train validation and test datasets. Training and validation datasets contain three columns (ID, Text, Label) while test dataset contains only 2 columns (ID, Text).

Furthermore, null values and duplicate rows are examined. Observing Table 3, we notice that there are no missing or duplicate values in both training and validation datasets.

2.1.2. Distribution of sentiment labels. In this step, the distribution of sentiment labels are examined to check for any dataset imbalance. Figure 1 shows that both train-

Dataset	Dimensions
Train	(148388, 3)
Validation	(42396, 3)
Test	(21199, 2)

Table 2: Dimensions of Datasets

Checks	Train Set	Validation Set
Missing Values in ID	0	0
Missing Values in Text	0	0
Missing Values in Label	0	0
Duplicate Rows	0	0

Table 3: Missing Values and Duplicate Rows Check

ing and validation datasets have equal percentage of positive and negative sentiment labels.

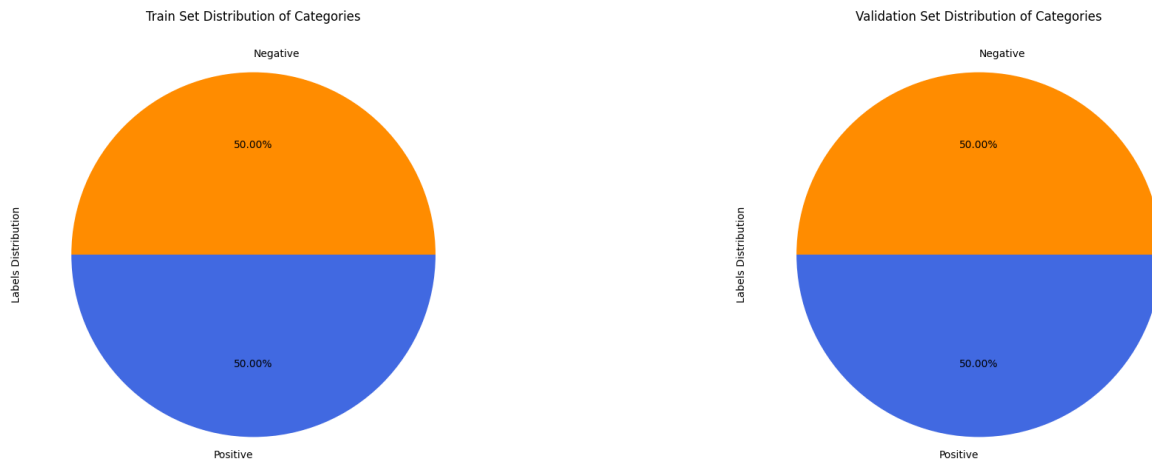


Figure 1: Distribution of tweet sentiment labels in the Training (left) and Validation (right) datasets.

2.1.3. Distribution of sentiment labels. Furthermore, we show the most common words before applying any preprocessing in the training dataset using word clouds [1]. Figure 2 shows the overall distribution of words using the entire training dataset. The second figure, Figure 3, presents two separate word clouds: one for tweets with positive sentiment and another for tweets with negative sentiment. These plots help visualize the most common keywords linked with each sentiment label. Some non-English characters were also found and will be removed later during the tweet preprocessing phase of the datasets.

In the positive sentiment cloud, we notice frequent use of words such as *love*, *thank*, and *good*. In contrast, the negative sentiment cloud contains words like *work*, *now*, and *going*.



Figure 2: Word cloud using entire training dataset.

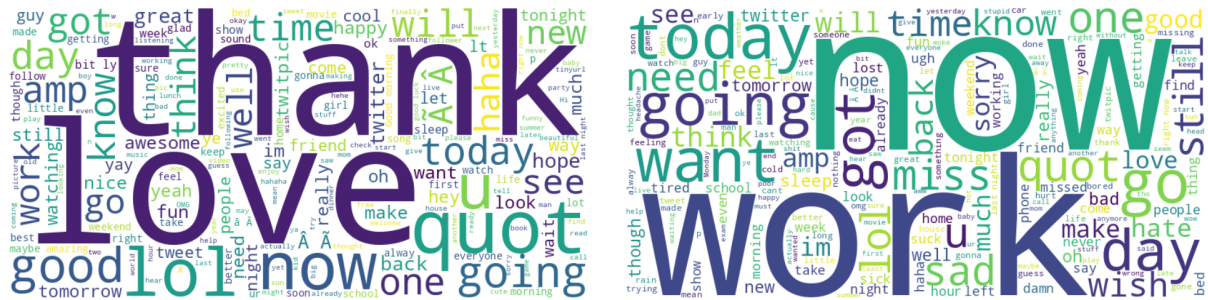


Figure 3: Word cloud representations of training set tweets: Positive sentiment (left) and Negative sentiment (right).

2.1.4. Tweets length analysis before preprocessing. Observing Figure 4 we notice that most tweets contain between 5 - 15 words with an average of 13.29 words per text. Moreover, Figure 5 shows the minimum and maximum length of tweets in training dataset before applying any preprocessing step.

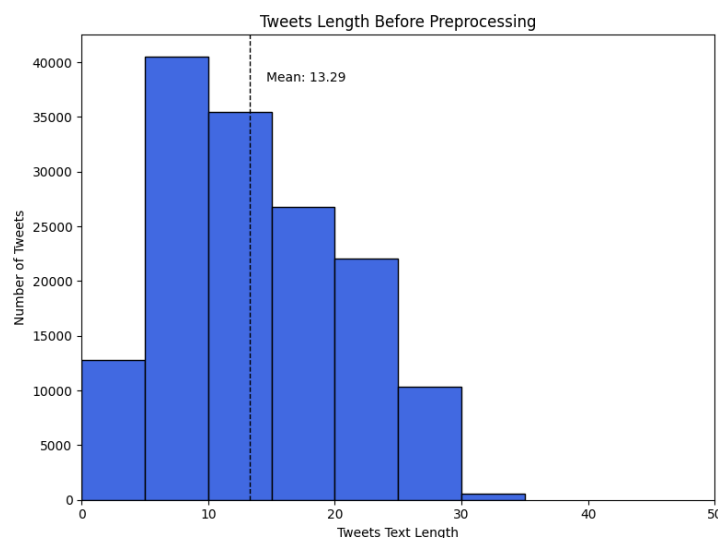


Figure 4: Distribution of tweet lengths in the dataset.

We preserve all the tweets even if the text length is one (but NOT white space character tweets in train and validation sets **only**).

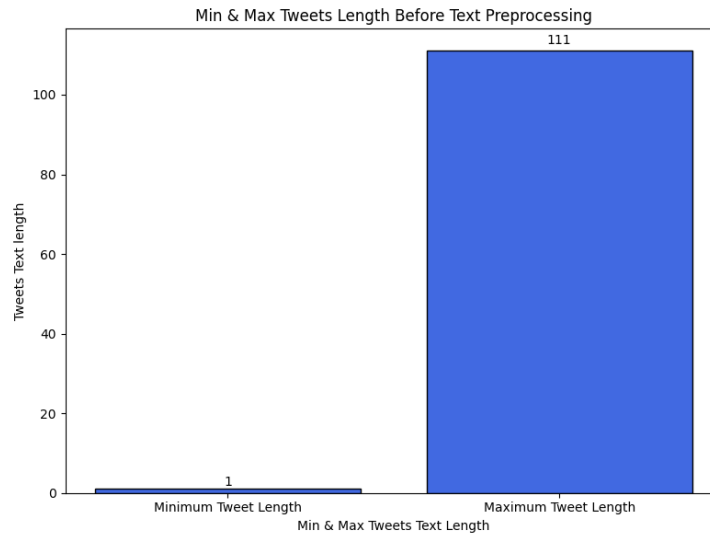


Figure 5: Minimum and maximum length of tweets in training dataset before applying text preprocessing.

2.1.5. Finding top 10 unigrams, bigrams & trigrams before preprocessing. In this step, we analyze the most frequent unigrams, bigrams, and trigrams from the dataset before applying any preprocessing. These n-grams provide insight into the most common word combinations and phrases used in tweets. Figures 6 and 7 show that most unigrams and bigram tokens are common stop words that may not contribute in our sentiment analysis.

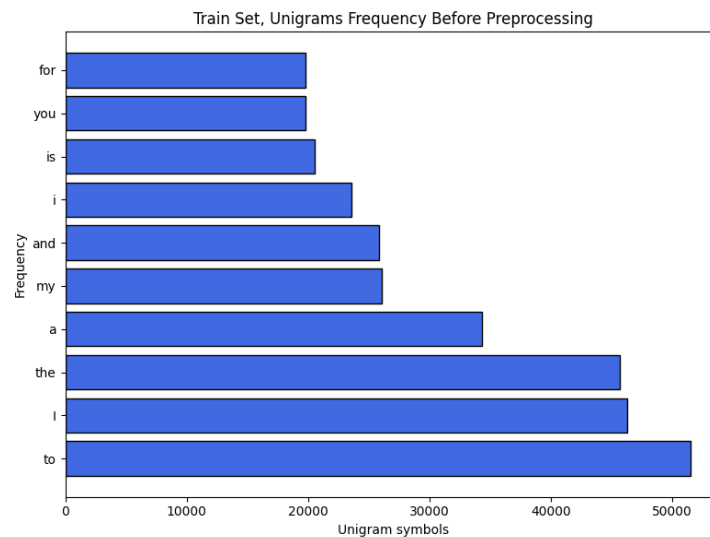


Figure 6: Top 10 unigrams in training set before applying text preprocessing.

2.1.6. Counting emails and user mentions in tweets. Figure 8 shows that a significant number of user mentions are consisted in tweets while the frequency of emails is low. In this project, during preprocessing, these tokens will be completely removed from the tweets compared to the first project where we anonymized them for feature extraction using TF-IDF vectorization and for privacy reasons. Moreover, experiments were

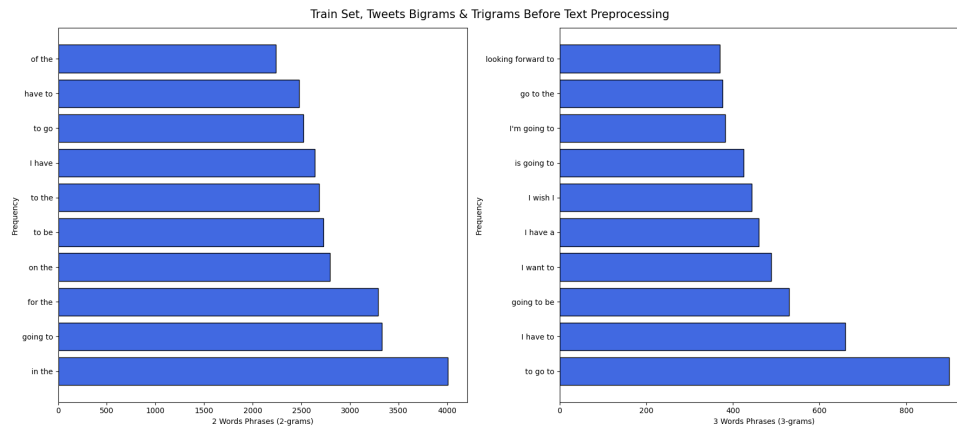


Figure 7: Top 10 bigrams and trigrams in training set before applying text preprocessing.

initially performed by preserving these tokens inside the dataset, and no better score results were noticed.

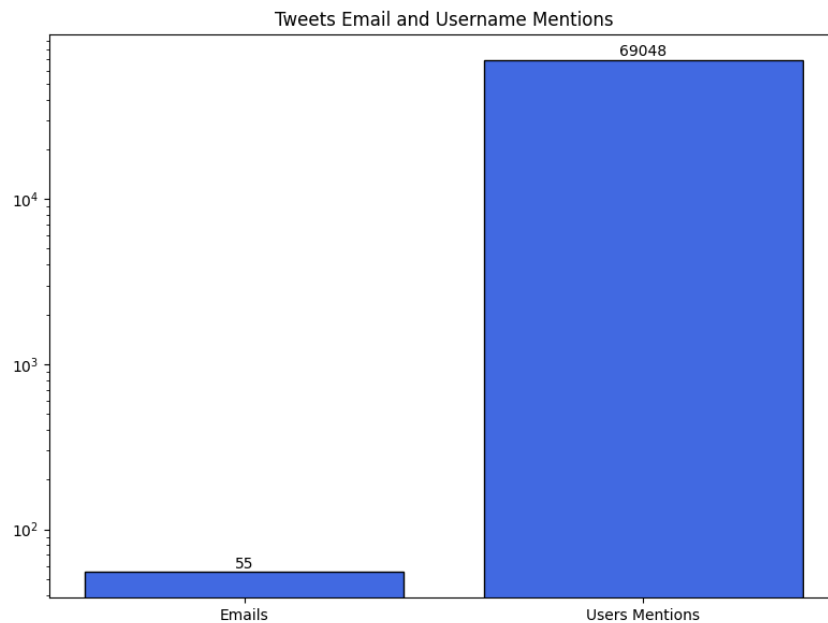


Figure 8: Counting email and user mentions in tweets before preprocessing on training dataset.

2.2. Tweets preprocessing & further analysis

Text preprocessing is a fundamental technique in natural language processing (NLP) that aims to clean the text by applying a series of transformations such as lowercasing, token elimination, and token conversion. These steps reduce noise, help the model focus on the most important tokens leading to better accuracy and generalization overall. Compared to previous projects, we slightly modified the preprocessing process by applying fewer steps to our tweets' dataset. These changes were motivated by experimental observations. Due to the specific tokenization process and the self-attention

mechanism used in the models, excessive preprocessing led to reduced performance on these models. Later, we will explore how internet slang (e.g., converting *lol* to *laughing out loud*) and stopwords removal affect the performance of these pretrained models.

A comprehensive pre-process approach is applied on tweets [5, 2, 4, 3, 8]. The preprocessing steps applied on all the datasets (train, validation, test) are the following:

- **Removing Non-ASCII Characters:** Non-ASCII characters are removed from tweets.
- **Lowercasing:** All characters in the text are converted to lowercase.
- **Expanding Contractions:** Shortened words are expanded to their full forms without apostrophes (e.g., *didn't* becomes *did not*).
- **Removing Hashtags:** Hashtag symbols are removed from our dataset, yet the hashtag's token is preserved. This gave better results due to the contextualized value these words give to our models.
- **Removing Mentions:** User mentions (e.g., @john) are now removed, compared to the first project where these tokens were anonymized using a @user token.
- **Removing Email Addresses:** Email addresses are removed, while in the first project they were anonymized with a XXX@email.com token.
- **Filtering Stop Words:** If stop word filtering is enabled, common stop words are removed from the text.
- **Filtering/Expanding Slang Words:** Common slang terms are detected and replaced [8] with their full meanings to improve data consistency (e.g., *lol* becomes *laughing out loud*).
- **Removing Empty Rows:** Any row with zero text length after applying the above steps is removed (applied only to the training and validation sets).

Compared to the previous two projects, removing URLs, digits, punctuation and applying lemmatization were also tested before being removed entirely, but it consistently resulted in lower validation scores for both of our models in all the experiments.

Tweet preprocessing experiments on our pretrained models are conducted later in three phases before selecting the best configuration for each one:

- Preprocessing with stopword filtering enabled and slang conversion disabled.
- Preprocessing with stopword filtering disabled and slang conversion disabled.
- Preprocessing with stopword filtering disabled and slang conversion enabled.

After the best preprocessing configuration is found we continue with the tokenization and model hyperparameter experiments. **For our data analysis preprocessing is done with stopwords filtering and slang conversion enabled to retrieve the *cleanest* dataset.**

2.2.1. Tweets length after preprocessing with stop words filtering enabled. Due to the removal of stop words, the length of tweets has decreased. Observing Figure 9, we notice that most tweets contain between 5 - 15 words with a mean value of 9.28. Furthermore, Figure 10 shows the minimum and maximum length of tweets in the training dataset after applying preprocessing. We notice that the maximum length of tweets has decreased from 111 to 69.

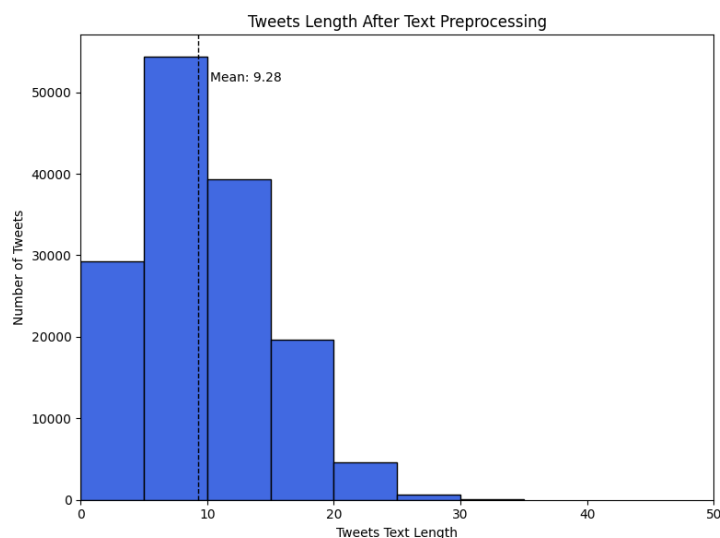


Figure 9: Distribution of tweet lengths in the dataset after applying preprocessing with filtering stop words enabled and slang words conversion enabled.

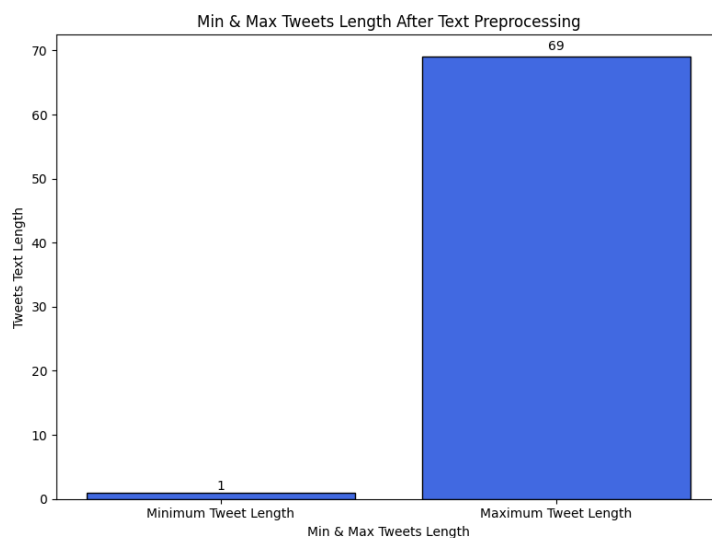


Figure 10: Minimum and maximum length of tweets in training dataset after applying text preprocessing with filtering stop words enabled and slang words conversion enabled.

2.2.2. Distribution of sentiment labels after applying preprocessing. We also show the most common words in the training dataset using wordclouds after running preprocessing with stop words filtering and slang words conversion enabled. Figure 11,

shows the overall distribution of words from the entire training dataset. The second figure, Figure 12, presents two separate word clouds: one for tweets with positive sentiment and another for tweets with negative sentiment.

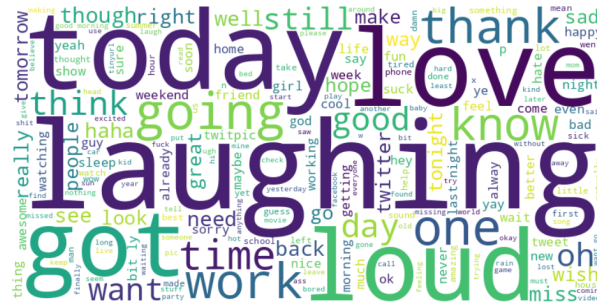


Figure 11: Word cloud using entire training dataset.

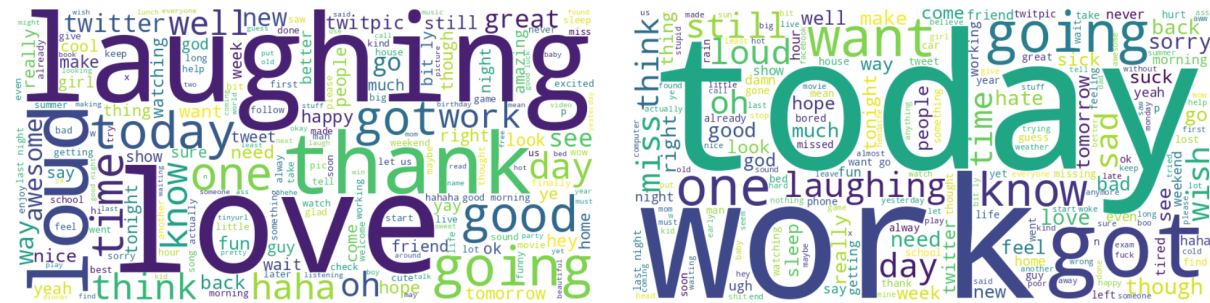


Figure 12: Word cloud representations of training set tweets after preprocessing: Positive sentiment (left) and Negative sentiment (right).

After examining the word clouds with preprocessing enabled, we notice that the most common words are *today*, *love* and *laughing* overall. In addition, within the positive data, pleasant words such as *love*, *thank* and *laughing* remain high in frequency. Finally, tweets with negative sentiment analysis highlight words such as *today*, *got* and *work*.

2.2.3. Finding top 10 unigrams, bigrams & trigrams after applying preprocessing. Figures 13, 14, 15, and 16 show a much clearer and more informative representation of the most frequent words compared to the non-preprocessed plots. This improvement is due to the removal of common stop words and slang words conversion, which often dominate the words distribution without adding important semantic value, though in our case we will later see that, similarly to the previous projects, by preserving stop words into the preprocessed dataset it provides better validation set accuracy with our pretrained models.

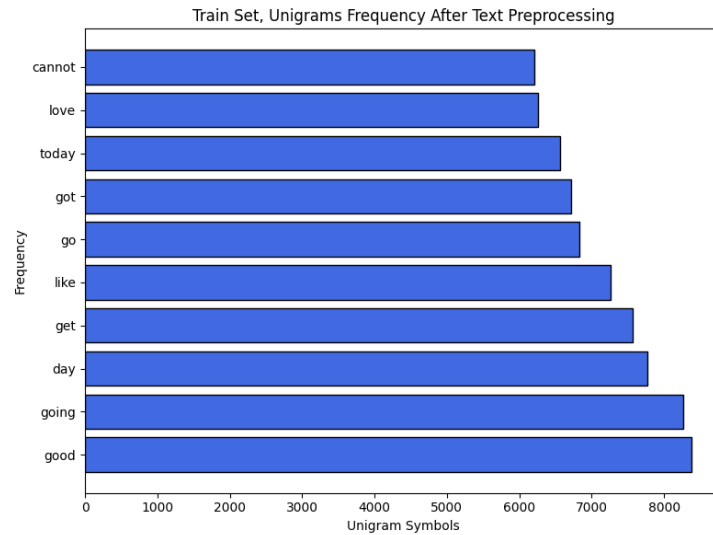


Figure 13: Top 10 unigrams in training set after applying tweets preprocessing.

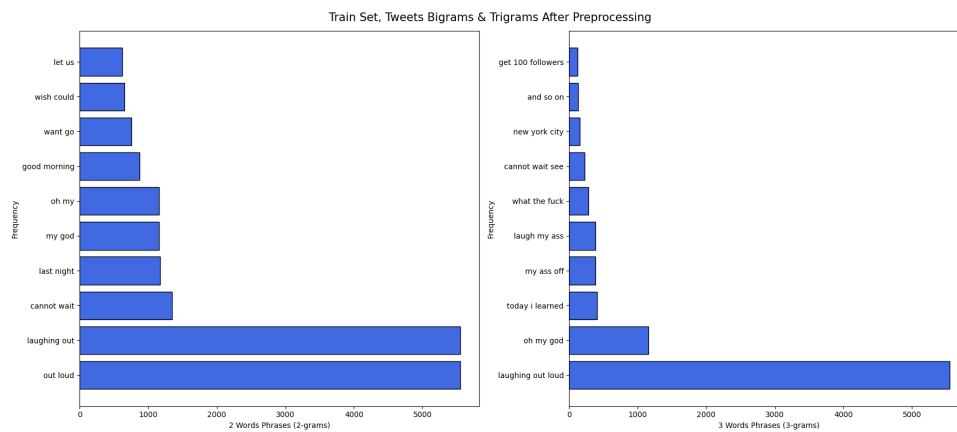


Figure 14: Top 10 bigrams and trigrams in training set after applying tweets preprocessing.

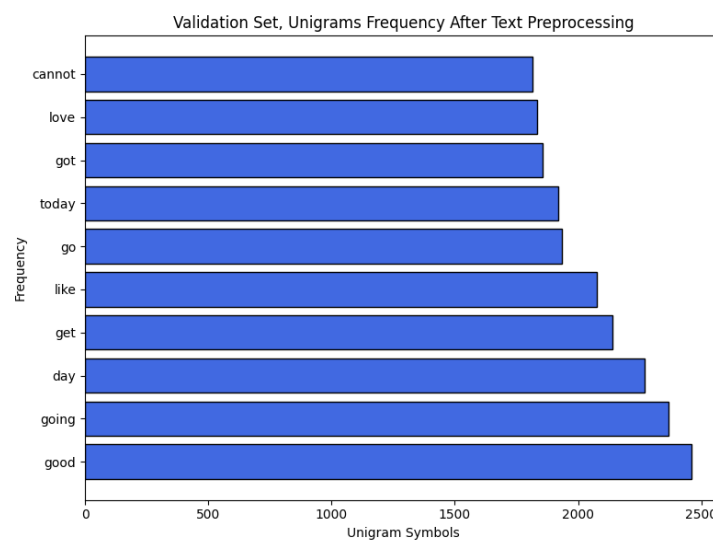


Figure 15: Top 10 unigrams in validation set after applying tweets preprocessing.

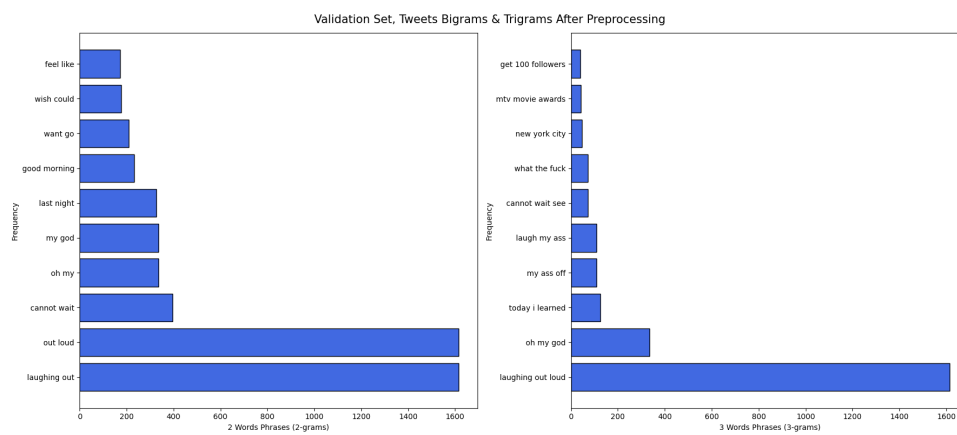


Figure 16: Top 10 bigrams and trigrams in validation set after applying tweets preprocessing.

3. Preprocessing and Tokenization Configurations

Before proceeding with the pretrained models hyperparameter tuning, multiple experiments were conducted using different preprocessing configurations alongside the tokenizers of the pretrained language models (BERT and DistilBERT) with the default parameter values. Then, after choosing the best preprocessing configuration for these two models we continue by searching the optimal max length for our model tokenizers. All these experiments are presented in detail in Section 4 alongside the hyperparameters fine-tuning of the models. In this Section we do a quick overview of our main logic of the experiments before the hyperparameters tuning.

3.1. Dataset preprocessing configurations

The experiments were conducted on two differently preprocessed dataset configurations:

- A preprocessed dataset with filtered stop words.
- A preprocessed dataset with preserved stop words.

The following acronyms are used to refer to the above configurations:

- **PRFS**: Preprocessing & Filtering Stopwords
- **PRPS**: Preprocessing & Preserving Stopwords

After determining that preserving stop words led to better performance across both models, we further continue our experiments by **enabling and disabling slang word conversion**, in order to find the best preprocessing configuration for our dataset in each model.

As we will show in Section 4, we notice that the configuration with preserved stop words provided better accuracy results overall on our validation set on both of our models. However, the two models do not perform equally well when slang words conversion is used.

3.2. Tokenization experiments

After selecting the best preprocessing configuration for each model we continue by analyzing the length distribution of the tweets based on the chosen configurations. We experiment with token lengths close to the tweets length such as 64, 128 and 256 tokens. For each of the above tokenization lengths, we evaluated performance and selected the one that had the highest validation accuracy at the epoch with the lowest validation loss. Finally after this step, we continue by experimenting on each model's hyperparameters and as a last step we apply an *Optuna* [11] study to search whether there is any addition optimization to our models.

3.3. Performance evaluation metrics

The performance evaluation metrics used during the training of these models on each epoch are the following:

- **Train Loss:** Measures the loss in the training dataset.
- **Validation Loss:** Measures the loss in the validation dataset.
- **Train Accuracy:** Measures the proportion of correctly classified instances in the training set.
- **Validation Accuracy:** Measures the proportion of correctly classified instances in the validation set.
- **Precision:** Measures the accuracy of positive predictions.
- **Recall:** Measures the proportion of actual positives that were predicted correctly as positives.
- **F1 Score:** Balances precision and recall and is mainly used for imbalanced datasets detection.

The above performance evaluation scores are evaluated at each epoch to guide us during the training & validation phase of the BERT and DistilBERT models.

4. Experiments on BERT and DistilBERT Language Models

This Section shows the experimental results for the BERT and DistilBERT [6, 7, 9, 13, 16] pretrained language models. These models are fine-tuned on preprocessing, tokenization, and model hyperparameter tuning. As described in the previous Section 3, we compare models' performance using accuracy, precision, recall, F1-score, and validation/test loss metrics. We first experiment with different preprocessing configurations, followed by adjustments to the tokenization maximum length, and finally tune the models' hyperparameters (optimizers, batch size, weight decay, learning rate, and eps). This experimentation was done by manually running each model with the corresponding parameters, and after finding a model baseline, we continued with some Optuna studies. All these hyperparameter fine-tuning experiments were conducted on 25% of the training and validation sets using sampling (with specific seeds) to speed up the process due to GPU limitations on the Kaggle platform. This also allowed us to explore larger variety of hyperparameters while keeping the classification labels percentage equal. In each section, we experiment and select the best hyperparameter, which is then used in the following section to progressively build toward a final BERT and DistilBERT model. Once the final hyperparameters were found for each model, we trained them on the **full training dataset** and evaluated the models on the entire validation and testing datasets, displaying the concluding results. **During our experiments and final training, we stored the model state (epoch checkpoint) with the lowest validation loss for each configuration and compared these saved states using validation accuracy and other scores to identify the most generalized model that performs well on both the validation and test datasets [12].**

Table 4 shows the initial model parameters that we will use for the experiments before running hyperparameter fine-tuning.

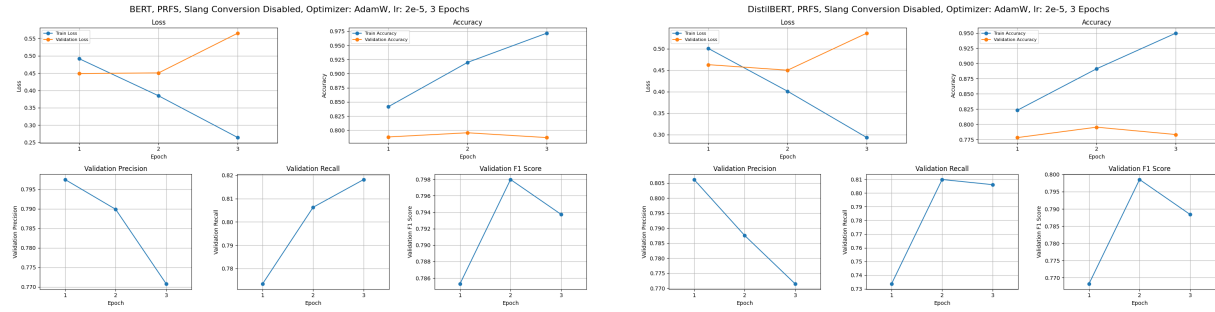
Model	BERT	DistilBERT
Preprocessing		
Tokenizer Max Length	128	128
Hyperparameters		
Training Batch Size	32	32
Validation Batch Size	64	64
Optimizer	AdamW	AdamW
Learning Rate	2×10^{-5}	2×10^{-5}
Weight Decay	0.0	0.0
Epsilon (eps)	1×10^{-8} (default)	1×10^{-8} (default)
Scheduler	None	None
Epochs	3	3

Table 4: Initial Preprocessing and training/validation hyperparameters of our two models.

4.1. Preprocessing comparison: Stopwords filtering and slang words conversion

As described in Section 3, the first set of experiments focuses on comparing different preprocessing strategies, specifically the impact of stopwords filtering (PRFS) versus stopwords preservation (PRPS), as well as the effect of enabling or disabling slang word conversion. These experiments were conducted to determine the most effective preprocessing configuration for both models before proceeding to the next stages of tokenization and hyperparameter tuning.

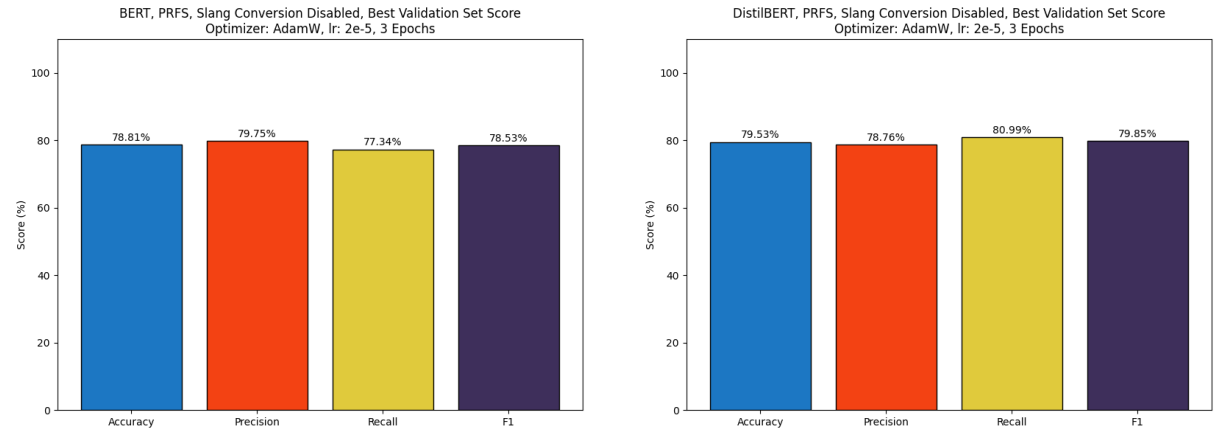
Figures 17, 18, 19, 20, 21, and 22 show the performance of BERT and DistilBERT models using the three preprocessing configurations described previously. We notice that overall, BERT achieves higher validation set scores and lower validation loss compared to DistilBERT. Overall this will also be the behavior between these two models in the following results. In addition, we observe that only one epoch is required to train our models. As training for more than one epoch introduces overfitting issues, as the validation loss gradually increases and the experimental generalization gap between validation and training set accuracy scores begins to widen. Finally, we also note that the remaining evaluation metrics are slightly unstable when models are trained for more than one epoch. Note, as we said earlier that fine-tuning is performed on 25% of the training and validation set, however, as we will see later, this behavior does not change when these models are trained and evaluated on the full datasets. **Based on the above results, we continue by selecting stop words preservation (PRPS) for both of the models. For the BERT model, we convert slang word acronyms (83.92% validation accuracy), but on DistilBERT we keep slang words acronyms conversion disabled due to slightly lower validation score in our experiments (83.06% validation accuracy).**



(a) BERT, Preprocessing with filtered stop words (PRFS), slang words conversion disabled.

(b) DistilBERT, Preprocessing with filtered stop words (PRFS), slang words conversion disabled.

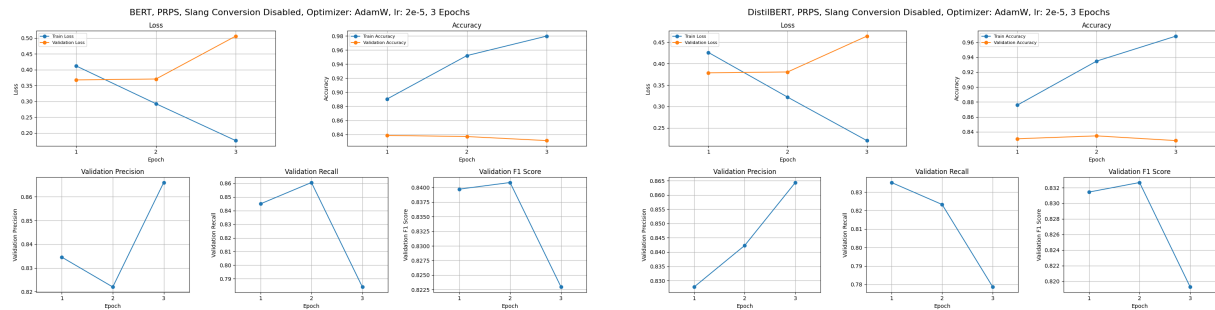
Figure 17: Comparison of learning rate preprocessing PRFS with slang words conversion step disabled on BERT and DistilBERT models.



(a) BERT, Best validation set score, preprocessing with filtered stop words (PRFS), slang words conversion disabled.

(b) DistilBERT, Best validation set score, preprocessing with filtered stop words (PRFS), slang words conversion disabled.

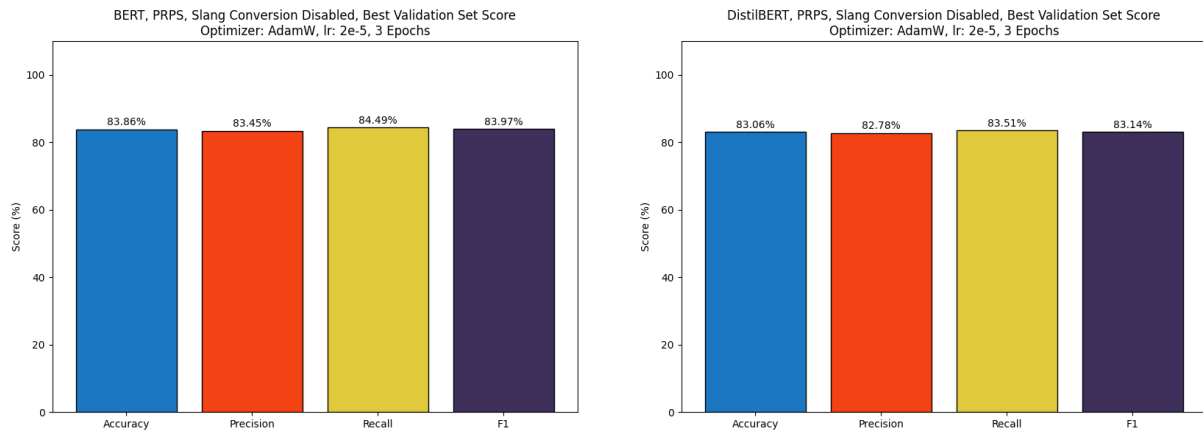
Figure 18: Comparison of best validation score on the lowest validation set loss, preprocessing PRFS with slang words conversion step disabled on BERT and DistilBERT models.



(a) BERT, Preprocessing with preserved stop words (PRPS), slang words conversion disabled.

(b) DistilBERT, Preprocessing with preserved stop words (PRPS), slang words conversion disabled.

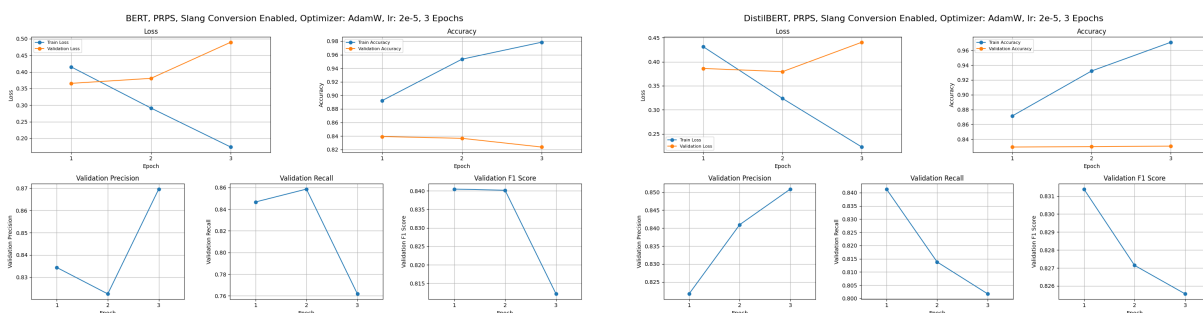
Figure 19: Comparison of learning rate preprocessing PRPS with slang words conversion step disabled on BERT and DistilBERT models.



(a) BERT, Best validation set score, preprocessing with filtered stop words (PRPS), slang words conversion disabled.

(b) DistilBERT, Best validation set score, preprocessing with filtered stop words (PRPS), slang words conversion disabled.

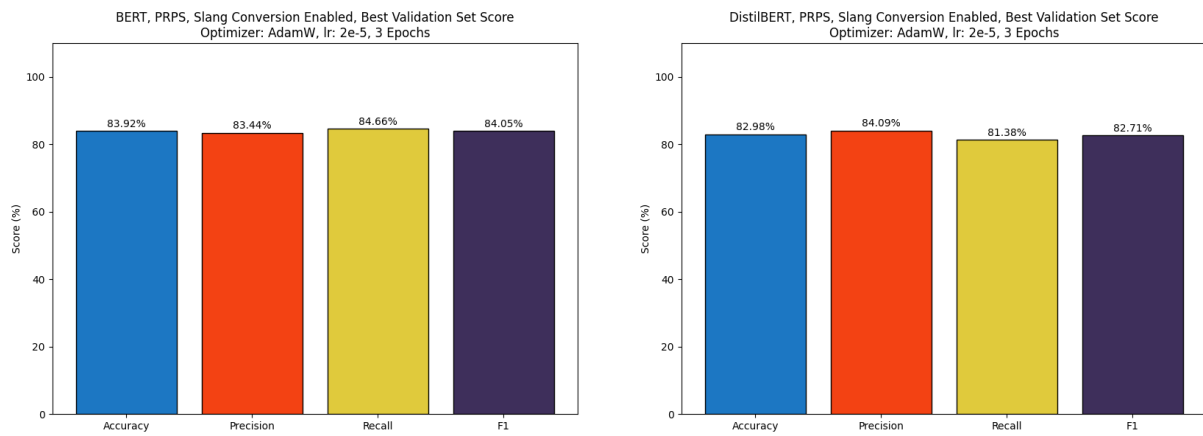
Figure 20: Comparison of best validation score on the lowest validation set loss, preprocessing PRPS with slang words conversion step disabled on BERT and DistilBERT models.



(a) BERT, Preprocessing with preserved stop words (PRPS), slang words conversion enabled.

(b) DistilBERT, Preprocessing with preserved stop words (PRPS), slang words conversion enabled.

Figure 21: Comparison of learning rate preprocessing PRPS with slang words conversion step enabled on BERT and DistilBERT models.



(a) BERT, Best validation set score, preprocessing with filtered stop words (PRPS), slang words conversion enabled.

(b) DistilBERT, Best validation set score, preprocessing with filtered stop words (PRPS), slang words conversion enabled.

Figure 22: Comparison of best validation score on the lowest validation set loss, preprocessing PRPS with slang words conversion step enabled on BERT and DistilBERT models.

4.2. Tokenization max length tuning

Choosing the maximum token length is an important step in fine-tuning transformer models as it directly affects both score performance and computational overhead. Since tweets are generally short, setting the tokenization length too high may introduce unnecessary padding, while setting it too low can lead to losing contextual data information. To guide this decision, we analyze the distribution of tweet lengths in the dataset and include a plot for visualization. In addition, based on this plot, we experiment with maximum token lengths of 64, 128, and 256 to assess how the input length affects the performance of our models. Figure 23 shows the length distribution for both tokenizers. We show two plots due to the difference in slang conversion preprocessing between the models, though notice that eventually they have the same distribution results. The maximum length of a tweet for both tokenizers are 82.

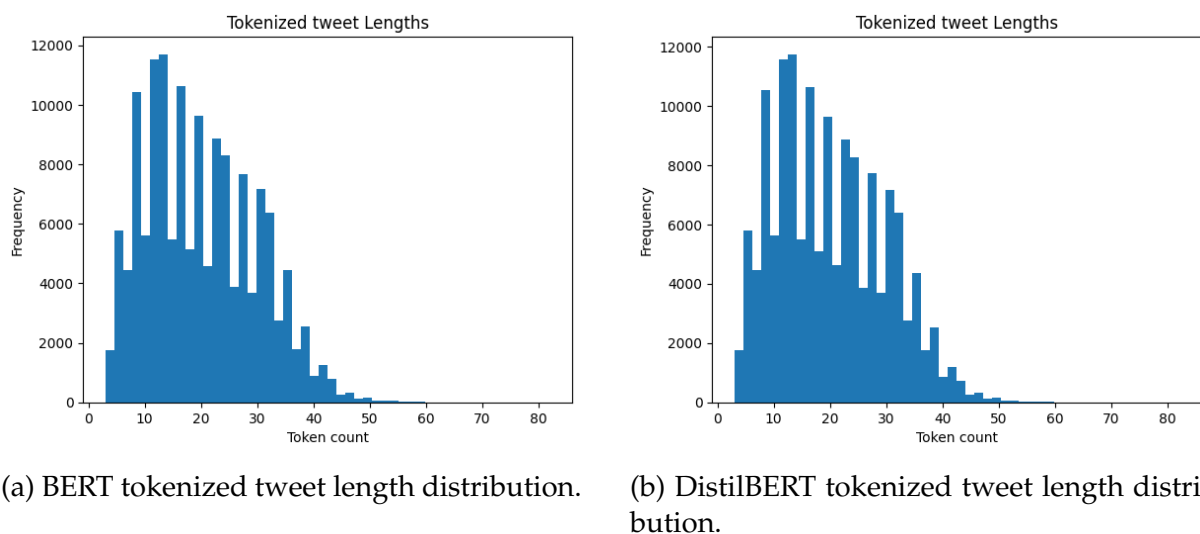
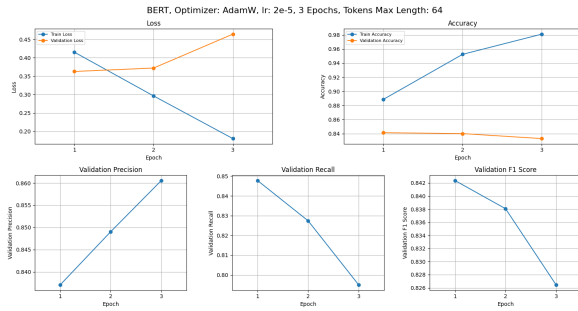
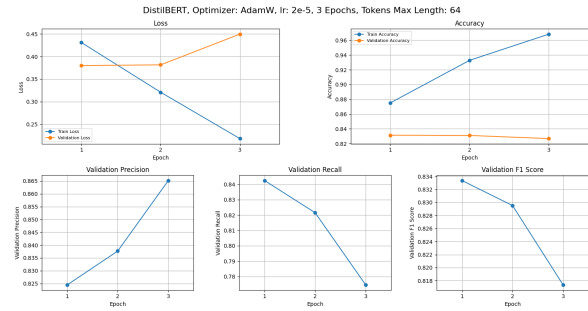


Figure 23: Tweet length distribution after tokenization for BERT and DistilBERT models.

Figures 24, 25, 26, 27, 28, and 29 show the training behavior and best validation scores of BERT and DistilBERT models using tokenization maximum sequence lengths of 64, 128, and 256 tokens. We observe that BERT achieves the highest validation accuracy with a maximum length of 64 tokens (84.13% validation accuracy), while DistilBERT performs best with a length of 128 tokens (83.34% validation accuracy). We observe similar behavior to validation set's precision, recall and f1 scores for both models. Again, only one epoch of training is enough to gain the lowest validation loss and not cause any overfitting issues. Therefore, since the best validation score is observed at the first epoch, we store a checkpoint at each epoch to select the most optimal model.

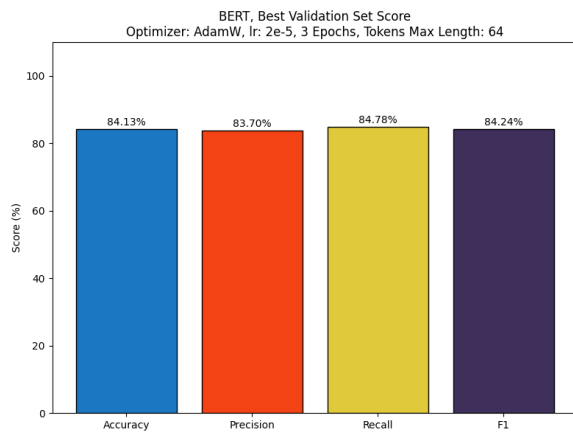


(a) BERT, learning rate curve, max length set to 64 tokens.

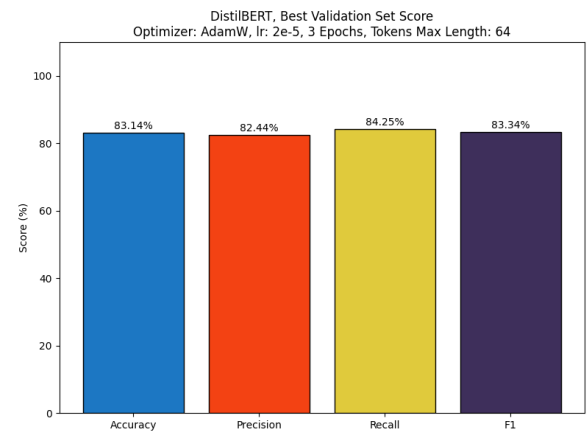


(b) DistilBERT, learning rate curve, max length set to 64 tokens.

Figure 24: Training behavior of BERT and DistilBERT with tokenization max length set to 64 tokens.

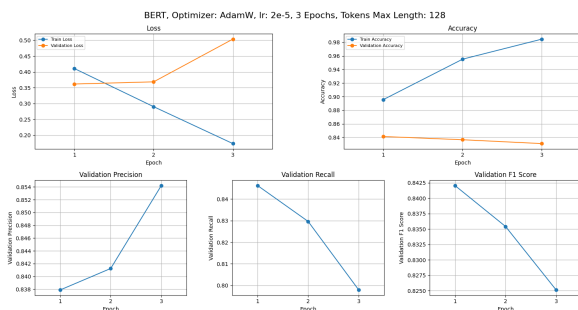


(a) BERT, best validation score, max length set to 64 tokens.

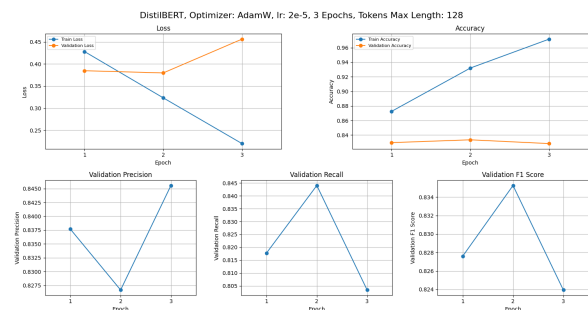


(b) DistilBERT, best validation score, max length set to 64 tokens.

Figure 25: Best validation set scores with tokenization max length set to 64 tokens.

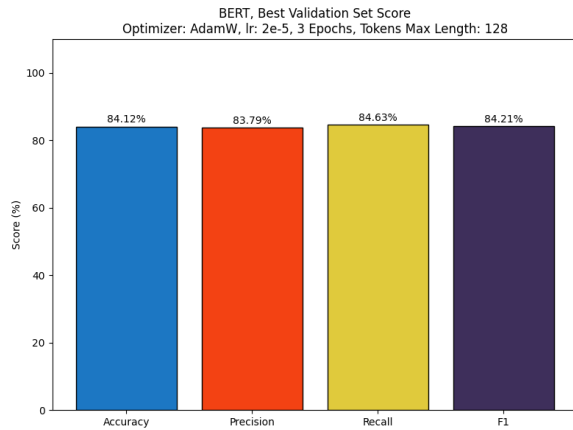


(a) BERT, learning rate curve, max length set to 128 tokens.

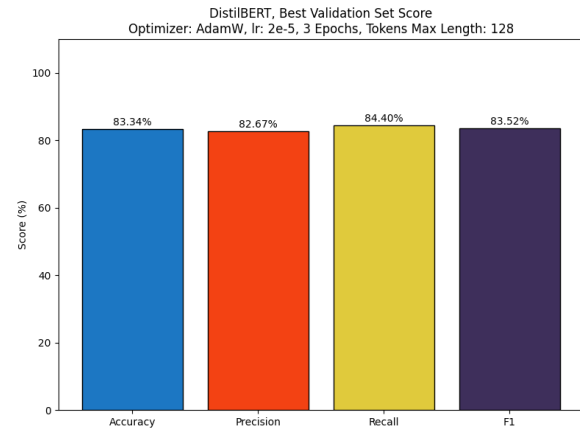


(b) DistilBERT, learning rate curve, max length set to 128 tokens.

Figure 26: Training behavior of BERT and DistilBERT with tokenization max length set to 128 tokens.

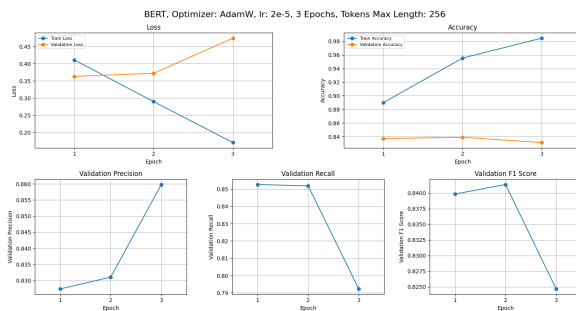


(a) BERT, best validation score, max length set to 128 tokens.

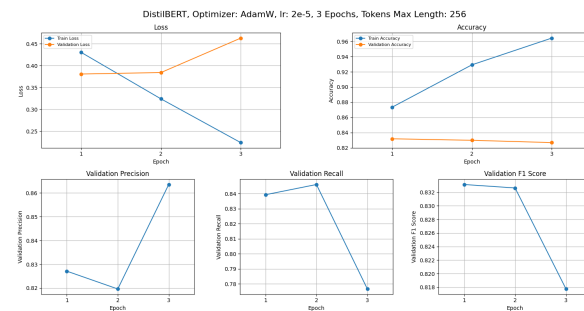


(b) DistilBERT, best validation score, max length set to 128 tokens.

Figure 27: Best validation set scores with tokenization max length set to 128 tokens.

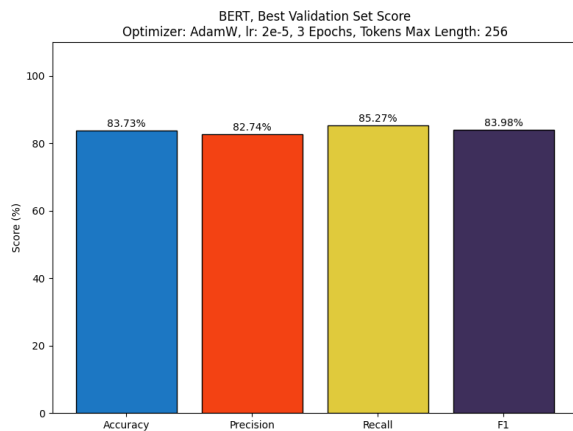


(a) BERT, learning rate curve, max length set to 256 tokens.

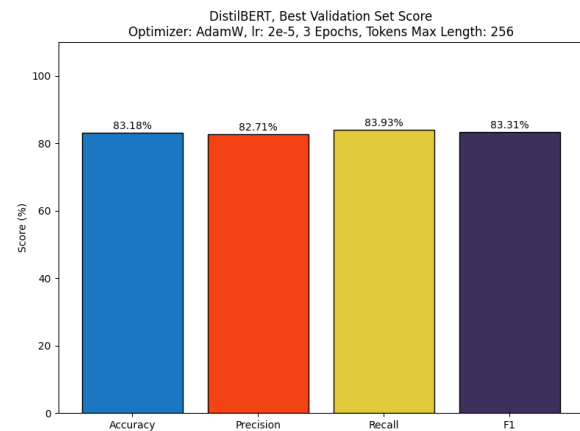


(b) DistilBERT, learning rate curve, max length set to 256 tokens.

Figure 28: Training behavior of BERT and DistilBERT with tokenization max length set to 256 tokens.



(a) BERT, best validation score, max length set to 256 tokens.



(b) DistilBERT, best validation score, max length set to 256 tokens.

Figure 29: Best validation set scores with tokenization max length set to 256 tokens.

4.3. Optimizer tuning

Optimizers are a very important component in neural network training, responsible for updating model parameters to minimize the loss function. In this section, we explore different optimizer configurations applied to both BERT and DistilBERT models. *AdamW* was used as the default optimizer in our baseline experiments. We now experiment with *Adam*, *NAdam*, *RAdam*, and *RMSprop*. Table 5 also shows the optimizers tested on both models. The figures below present the training dynamics and best validation performance for each optimizer and model. Figures 30, 31, 32, 33, 34, 35, 36, and 37 show the results of the optimizers. We observe that for BERT, *AdamW* (Figure 25) remains with the highest validation score, whereas for DistilBERT, the *Adam* optimizer provides the best result (83.43% compared to 83.34%). Validation loss, precision, recall and f1 scores remain nearly the same with the first *AdamW* optimizer.

Optimizers Tested
Adam
NAdam
RAdam
RMSprop

Table 5: List of tested optimizers on BERT and DistilBERT

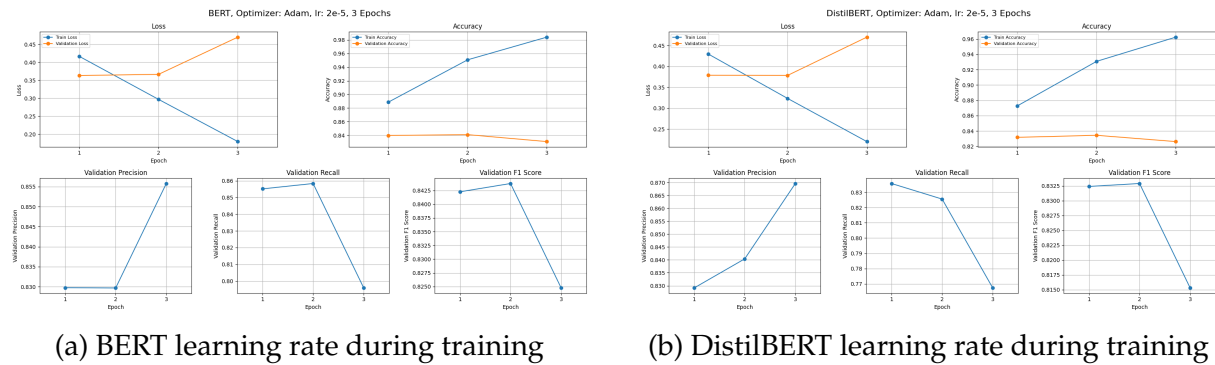
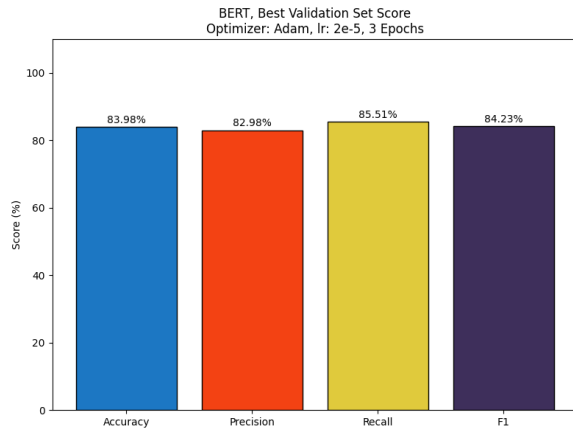


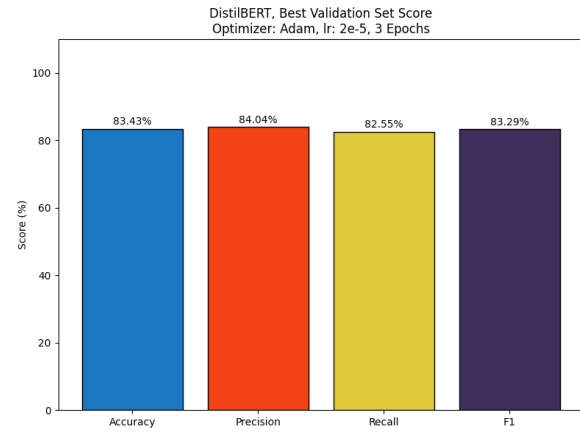
Figure 30: Learning rate curves using Adam optimizer

4.4. Optimizer scheduler testing

Finally in this section before continuing with Optuna studies, we experimented with the `get_linear_schedule_with_warmup` optimizer scheduler for both BERT and DistilBERT models. Figures 38 and 39 show the learning rate through the epochs training and the best validation score on the lowest validation loss epoch. We notice that the validation accuracy has slightly decreased for both models from 84.13% to 83.73% for BERT and from 83.43% to 83.17% for DistilBERT. Precision, recall and f1 scores remain almost the same with the previous experiments.

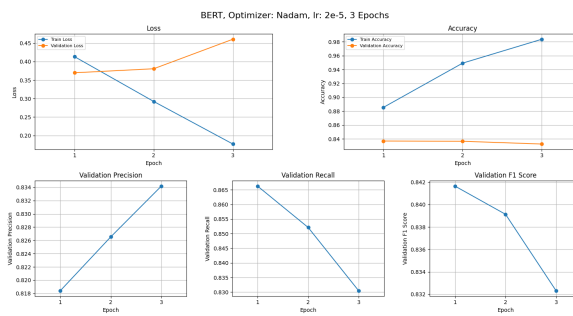


(a) BERT best validation score at lowest validation loss

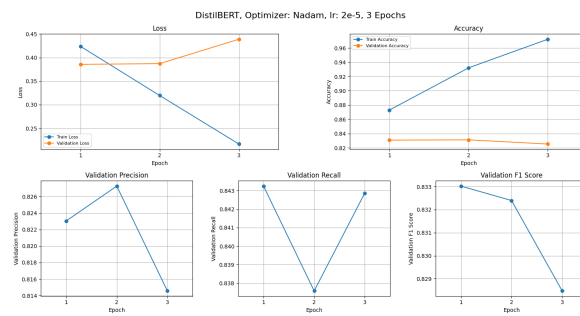


(b) DistilBERT best validation score at lowest validation loss epoch

Figure 31: Best validation scores using Adam optimizer

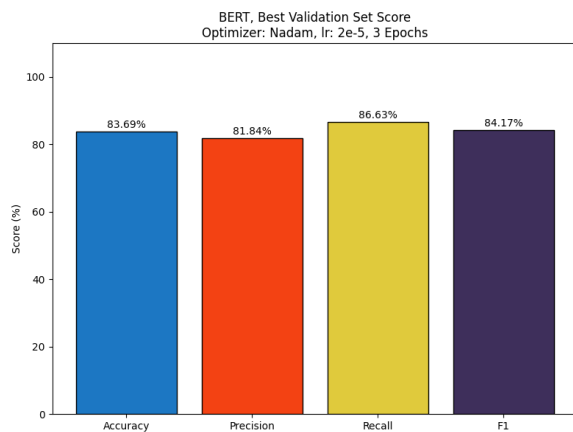


(a) BERT learning rate during training

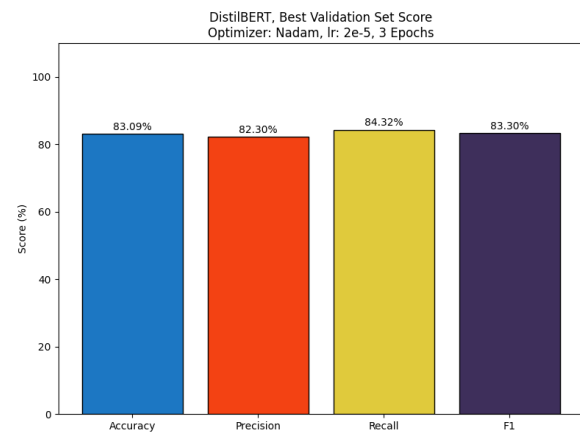


(b) DistilBERT learning rate during training

Figure 32: Learning rate curves using NAdam optimizer

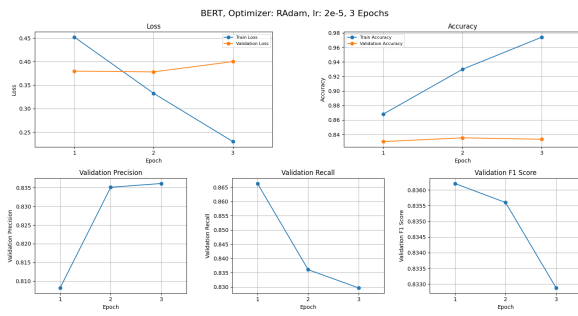


(a) BERT best validation score at lowest validation loss epoch

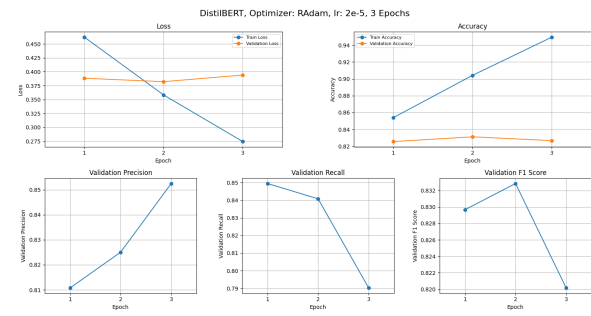


(b) DistilBERT best validation score at lowest validation loss epoch

Figure 33: Best validation scores using NAdam optimizer

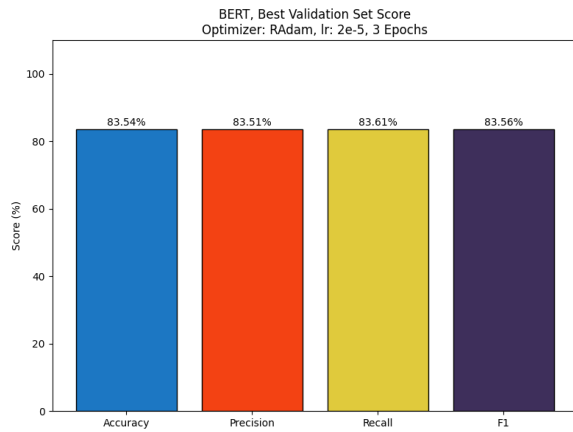


(a) BERT learning rate during training

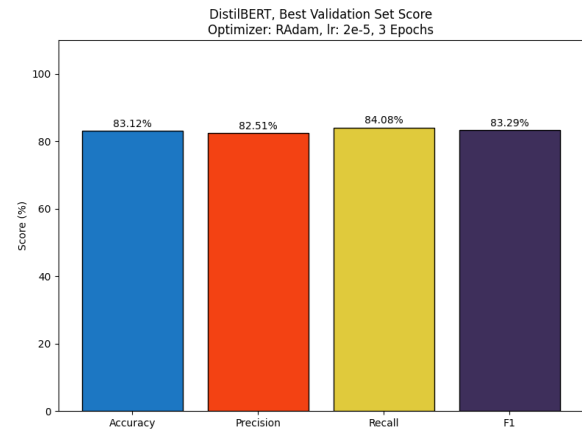


(b) DistilBERT learning rate during training

Figure 34: Learning rate curves using RAdam optimizer

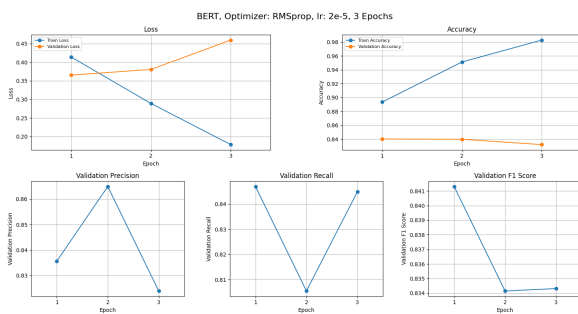


(a) BERT best validation score at lowest validation loss epoch

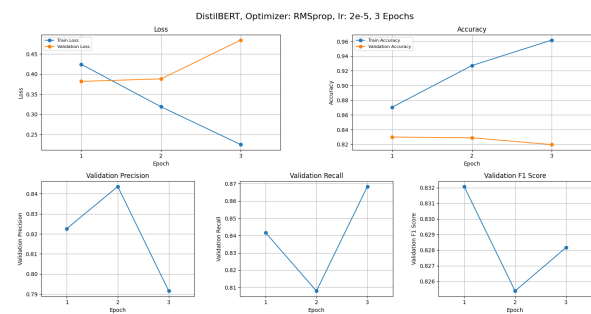


(b) DistilBERT best validation score at lowest validation loss epoch

Figure 35: Best validation scores using RAdam optimizer

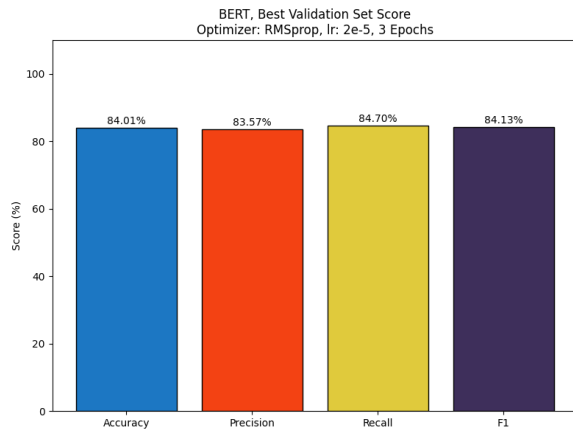


(a) BERT learning rate during training

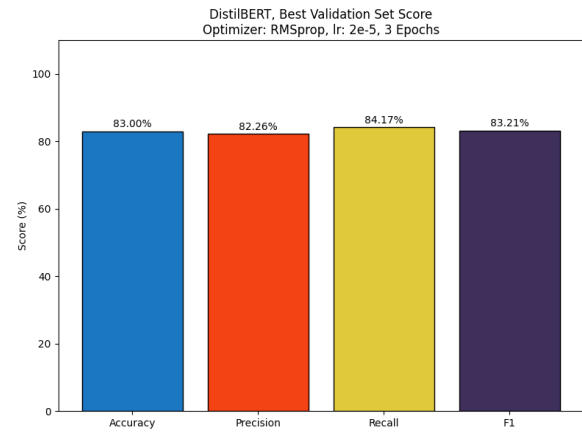


(b) DistilBERT learning rate during training

Figure 36: Learning rate curves using RMSprop optimizer

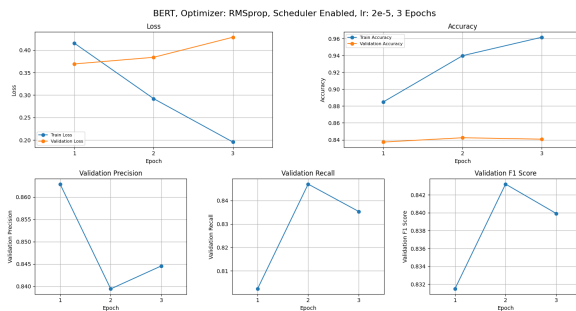


(a) BERT best validation score at lowest validation loss epoch

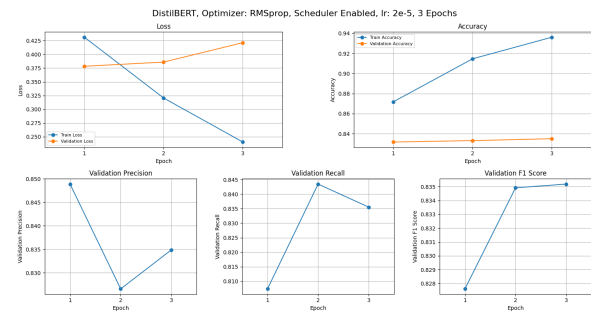


(b) DistilBERT best validation score at lowest validation loss epoch

Figure 37: Best validation scores using RMSprop optimizer

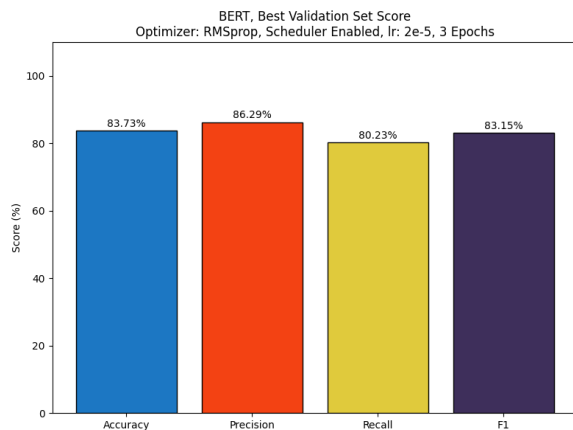


(a) BERT learning rate schedule (using AdamW)

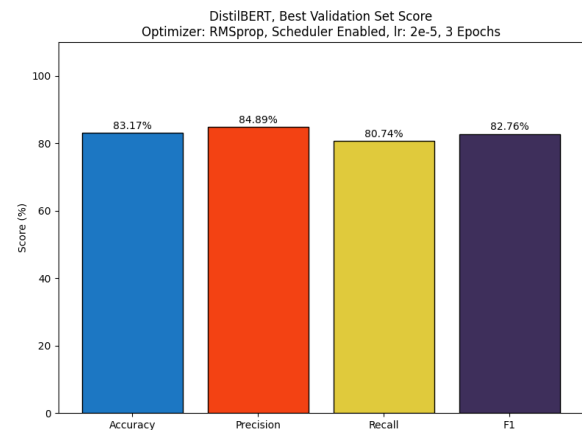


(b) DistilBERT learning rate schedule (using Adam)

Figure 38: Learning rate scheduler during training using the best optimizer



(a) BERT best epoch (lowest validation loss) (using AdamW)



(b) DistilBERT best epoch (lowest validation loss) (using AdamW)

Figure 39: Best epochs identified during training using learning rate schedulers

4.5. Optuna study

Finally, the best-performing hyperamaters configurations for each model based on preprocessing, tokenization, optimizers selection was used as the baseline for hyperparameter optimization using Optuna. For the DistilBERT model we ran 10 studies and for BERT we run 5 studies still on the 25% of our dataset due to GPU limitations. The search was conducted on parameters including weight decay, batch size, learning rate, and eps that we had not yet experimented with in our previous configurations. Table 6 shows the best model architecture and hyperparameter configuration so far before using Optuna optimization framework.

Component	BERT	DistilBERT
Preprocessing		
Max Length	64	128
Hyperparameters		
Batch Size	32	32
Optimizer	AdamW	Adam
Learning Rate	2×10^{-5}	2×10^{-5}
Weight Decay	0.0 (default)	0.0 (default)
Epsilon (eps)	1×10^{-8} (default)	1×10^{-8} (default)
Scheduler	None	None
Best Epoch	1	1

Table 6: Preprocessing and training hyperparameter configurations for BERT and DistilBERT before Optuna tuning.

We explored combinations of the following hyperparameters starting with the best configuration found thus far:

- Learning rates from 1×10^{-5} to 5×10^{-5} (log scale)
- Batch sizes from the set [16, 32]
- Weight decay values from 0.0 to 0.1 in increments of 0.01
- Epsilon values from 1×10^{-8} to 1×10^{-6} (log scale)

The goal of the Optuna study was to minimize the validation loss and find a better generalized model with improved validation accuracy.

The final model configuration after Optuna hyperparameter optimization is presented in Table 7. Notice that for BERT, Optuna identified improved hyperparameters that had better validation accuracy, however for DistilBERT, the tuning process did not give any parameters that outperformed our initial configuration.

Component	BERT	DistilBERT
Preprocessing		
Max Length	64	128
Training Hyperparameters (after Optuna tuning)		
Batch Size	32	32
Optimizer	AdamW	Adam
Learning Rate	1.34×10^{-5}	2×10^{-5}
Weight Decay	0.04	0.0
Epsilon (eps)	3.82×10^{-8}	1×10^{-8}
Scheduler	None	None
Best Epoch	1	1

Table 7: Preprocessing and training hyperparameter configurations for BERT and DistilBERT after Optuna tuning.

5. Final Results

In this section we evaluate our best models for BERT and DistilBERT using the best hyperparameters found from the previous experiments. Table 7 shows the final hyperparameters of the two models. We train and evaluate both models on the entire train and validation datasets, compare the final results and show their corresponding roc curves, confusion matrices [15, 14] as well as predicting the test set of the Kaggle competitions.

5.1. BERT final model results

Figures 40 show the final training and evaluation of BERT model on the entire training dataset, evaluating with the entire validation set on each epoch. We observe that training for one epoch is enough for our dataset, while continuing to train further introduces overfit issues as we see in Figure 40 (left) on training validation loss and accuracy plots. We also notice that precision, recall and f1 score become unstable though the epochs training. For the above reasons we save and load the best state of model which is on the first epoch, and the results are displayed in detail in Figure 40 (right). BERT model achieves 85.12% validation set accuracy on the validation dataset. Moreover, Figure 41 shows the ROC curve and confusion matrix of BERT model on the validation dataset. The AUC of 0.93 shows that the model has a solid classification performance and it is able to distinguish between positive and negative tweets. The confusion matrix shows us the distribution of misclassification presenting the false negative and false positive predictions of our final BERT model. We notice that the number of false positives and false negatives are almost identical in our validation set experiment. Finally, the test set (Kaggle submission) accuracy score for the BERT model is 85.178% which is very close to the validation set accuracy score and shows that the final model does not overfit to new unseen data.

5.2. DistilBERT final model results

Similarly, Figures 42 show the final training and evaluation of DistilBERT model on the entire training dataset, evaluating with the entire validation set on each epoch. We

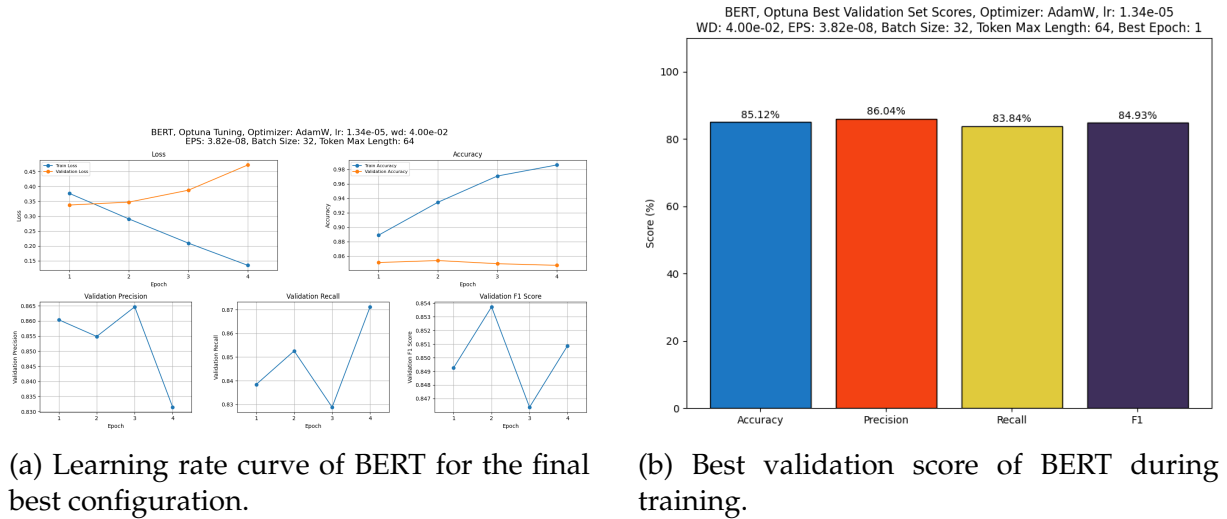


Figure 40: Final training behavior of BERT under best configuration.

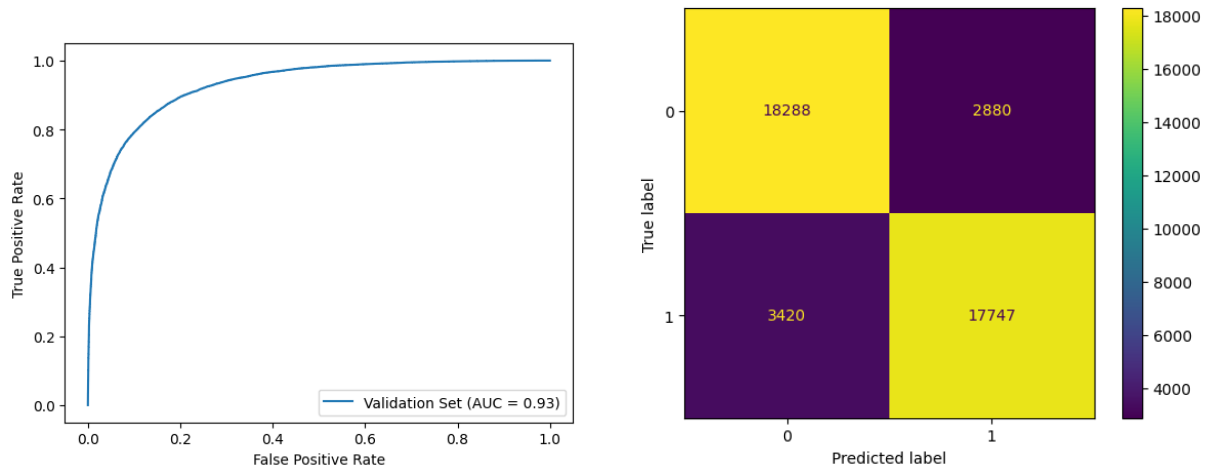


Figure 41: Final evaluation of BERT on the validation dataset.

observe that training for one epoch is enough for our dataset, while continuing to train further introduces again overfit issues as we see in Figure 42 (left) on training validation loss and accuracy plots. We also notice that precision, recall and f1 score become unstable though the epochs training. For the above reasons we save and load the best state of model which is on the first epoch, and the results are displayed in detail in Figure 42 (right). DistilBERT model achieves 84.36% validation set accuracy on the validation dataset. Moreover, Figure 43 shows the ROC curve and confusion matrix of BERT model on the validation dataset. The AUC of 0.92 like the BERT model shows that it has a very good classification performance and it is able to distinguish between positive and negative tweets. We notice again that the number of false positives and false negatives identical with the false negatives being slightly more in our validation set experiment. Finally, the test set (Kaggle submission) accuracy score for the DistilBERT model is 84.08% which is very close to the validation set accuracy score and shows again that the final model does not overfit to new unseen data.

Overall we notice that BERT has better validation scores than DistilBERT which is expected due to the larger number of parameters and layers, though it takes significantly more time to train and inference.

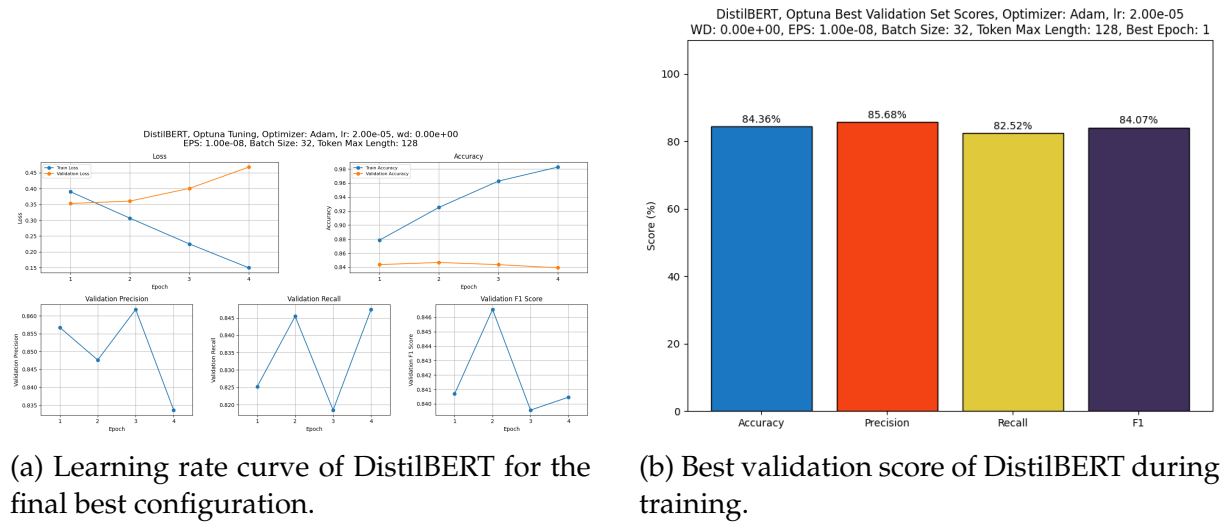


Figure 42: Final training behavior of DistilBERT under best configuration.

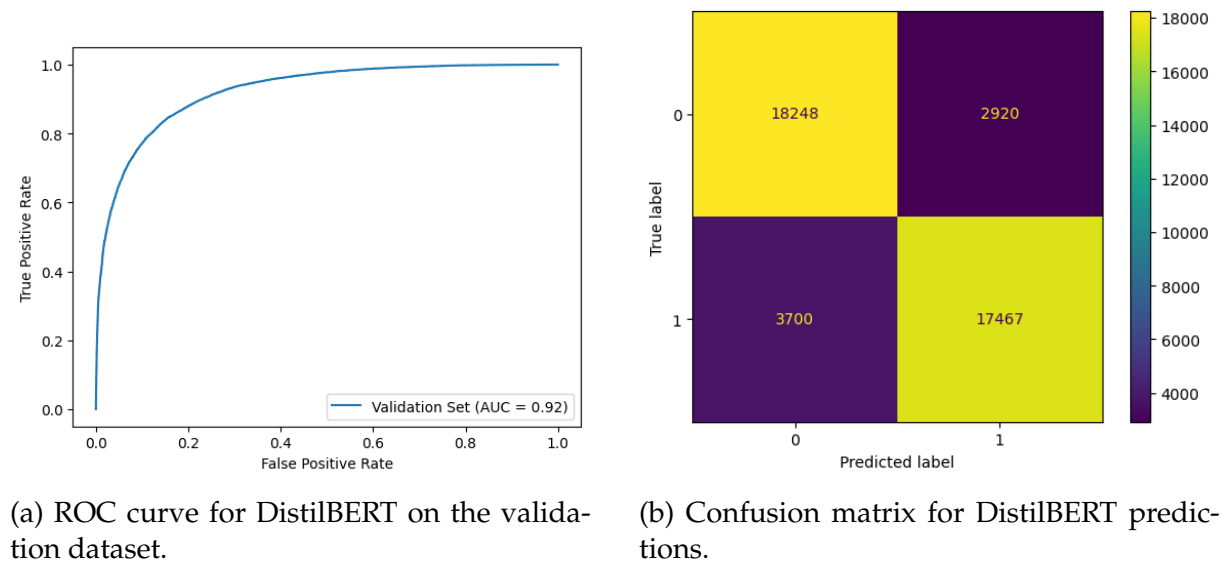


Figure 43: Final evaluation of DistilBERT on the validation dataset.

5.3. Comparison with the first project

Compared to the first project, which used TF-IDF for feature extraction and a logistic regression model, BERT and DistilBERT models show an improvement in performance from 80.37% to 85.12% and 84.36%. This increase is due to the pretrained architecture of the language models and the self-attention layers and the contextualized links between tokens. However, this performance increase has its drawback of longer training times and a larger number of hyperparameters that need to be fine-tuned.

5.4. Comparison with the second project

Compared to the second project, which used a feed-forward neural network on averaged Word2Vec embeddings with pretrained GoogleNews vectors, BERT and DistilBERT models again show a more significant improvement in performance from 77.88% to 85.12% and 84.36%, respectively. This performance gain is due to the pretrained nature of BERT and DistilBERT and their self-attention layers, which link contextual relationships between tokens unlike the average token embeddings used in the feedforward model. However, this improvement comes with the drawback of longer training times, though hyperparameter tuning is less extensive and has a smaller affect on the final performance of these models.

5.5. Future work

As future work, we would like to experiment with a larger number of Optuna studies and on a higher subset percentage of training and validation datasets during our fine-tuning experiments. Finally, we would also like to experiment with different learning rate schedulers.

6. Bibliography

References

- [1] amueller. Word cloud by andreas mueller. https://amueller.github.io/word_cloud/.
- [2] geeksforgeeks. Check if email address is valid or not in python. <https://www.geeksforgeeks.org/check-if-email-address-valid-or-not-in-python/>.
- [3] geeksforgeeks. Python nltk: Tweettokenizer. <https://www.geeksforgeeks.org/python-nltk-nltk-tweettokenizer/>.
- [4] geeksforgeeks. Python regex lookahead. <https://www.geeksforgeeks.org/python-regex-lookahead/>.
- [5] geeksforgeeks. Text preprocessing for nlp tasks. <https://www.geeksforgeeks.org/text-preprocessing-for-nlp-tasks/>.
- [6] google. Google: bert-base-uncased. <https://huggingface.co/google-bert/bert-base-uncased>.
- [7] google. Google: distilbert-base-uncased. <https://huggingface.co/distilbert/distilbert-base-uncased>.
- [8] kaggle. kaggle slang words for nlp. <https://www.kaggle.com/code/nmaguette/up-to-date-list-of-slangs-for-text-preprocessing>.
- [9] Piyush Kashyap. Guide to tokenization and padding with bert: Transforming text into machine-readable data. <https://medium.com/@piyushkashyap045/guide-to-tokenization-and-padding-with-bert-transforming-text-into-machine-readable-data->

- [10] matplotlib. matplotlib examples. <https://matplotlib.org/stable/gallery/index.html>.
- [11] optuna. Optuna pythonic search space. https://optuna.readthedocs.io/en/stable/tutorial/10_key_features/002_configurations.html.
- [12] Pytorch. Saving and loading models. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html.
- [13] Rayyan Shaikh. Mastering bert: A comprehensive guide from beginner to advanced in natural language processing (nlp). <https://medium.com/@shaikhrayyan123/a-comprehensive-guide-to-understanding-bert-from-beginners-to-advanced-2379699e2b51>.
- [14] sklearn. sklearn metrics confusion matrix. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html.
- [15] sklearn. sklearn metrics roc curve. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_curve.html.
- [16] Bradney Smith. A complete guide to bert with code. <https://medium.com/data-science/a-complete-guide-to-bert-with-code-9f87602e4a11>.